

SOFTWARE ENGINEERING

(Lab 4: VibeCoding)

PROMPT 1:

I am working on a Python terminal-based Connect Four project.

IMPORTANT:

- Do NOT rewrite the entire project.
- Do NOT replace the existing architecture.
- Keep the existing files: main.py, game.py, board.py and ai.py.
- Make only the changes necessary for this task.
- Do not add external dependencies or persistent storage.
- First inspect the existing code and explain briefly what is wrong before modifying it.

TASK 1 — COMPLETE WIN DETECTION

The current Board.winner() implementation checks horizontal and vertical four-in-a-row patterns, but it does not correctly detect diagonal wins.

Modify board.py so that winner(token) correctly detects all four directions:

1. Horizontal: left-to-right
2. Vertical: top-to-bottom
3. Diagonal: top-left to bottom-right
4. Diagonal: bottom-left to top-right

Requirements:

- Exactly four consecutive matching tokens must be detected as a win.
- Longer sequences containing four consecutive tokens should also count as a win.
- Three consecutive tokens must NOT count as a win.
- Do not break the existing horizontal or vertical detection.
- Keep the existing Board class and grid representation.
- Do not modify unrelated files unless absolutely necessary.

After making the change:

1. Show me exactly what you changed.
2. Explain the logic in simple terms.
3. Give me a short manual test plan for horizontal, vertical and both diagonal wins.

“The prompt instructs the AI to fix Connect Four win detection logic so it can recognize horizontal, vertical, and both diagonal four in a row patterns. It also ensures that the existing project structure is preserved and the changes are limited to what is necessary for Task 1.”

PROMPT 2:

Now continue working on the SAME Connect Four project and existing code.

Do NOT rewrite the project or change its architecture.

TASK 2 — COMPLETE GAME TERMINATION AND INVALID MOVE HANDLING

Improve the game flow so that all terminal and invalid states are handled consistently.

Requirements:

1. Full columns:

- If a player selects a column that is already full, do not change the board.
- Show a concise message that the column is unavailable.
- Allow the human player to choose another column.
- The game must not crash.

2. Invalid human input:

- Non-numeric input should not crash the game.
- A number outside columns 1–7 should be rejected safely.
- The board must not change.
- The player should be allowed to try again.

3. AI and full columns:

- If the AI is asked to move, it must never attempt an illegal/full column.
- If there are no legal columns, the game should end as a draw rather than crash or terminate unexpectedly.

4. Win termination:

- After a winning move, immediately announce the winner.
- Do NOT request another turn after a winning move.

5. Draw termination:

- If the board becomes full without a winner, announce a draw and end the game.

Keep the existing Game, Board and AI classes and make the smallest reasonable changes.

After modifying the code:

- Explain each change.
- Show me how each requirement can be manually tested.
- Do not add external dependencies or persistent storage.

“The prompt instructs the AI to improve the game's handling of invalid moves and game ending conditions, including full columns, invalid input, wins, and draws. It ensures the game stops correctly after a win or draw while keeping the existing project structure and avoiding unnecessary changes.”

PROMPT 3:

Continue working on the SAME Connect Four project.
Do not replace the project with a new implementation.
Keep the existing AI class and Board representation.

TASK 3 — IMPROVE THE AI

The current AI chooses a random legal column. Replace this random-only decision-making with meaningful decision-making.

The AI must follow this priority:

1. WIN:

If the AI has an immediate winning move available, choose that column.

2. BLOCK:

If the human player has an immediate winning move on their next turn, block that column.

3. OTHERWISE:

Choose any legal column safely.

Requirements:

- Never choose a full or invalid column.
- If there are no legal columns, return None.
- When checking a possible move, do not permanently modify the real board unless the move is correctly simulated and restored.
- The existing game must continue to work with the AI.
- Do not add external libraries.
- Keep the implementation understandable for a beginner.
- Do not add persistent storage.

Important:

The AI should detect an immediate winning move by simulating a move and checking whether that move creates four in a row.

Similarly, it should detect an immediate threat from X by simulating X's possible move and checking whether X would win.

After modifying the code:

1. Explain the AI decision process.
2. Show exactly what changed.
3. Give me manual test scenarios for:
 - AI immediate win
 - AI immediate block
 - full columns
 - no legal columns
4. Check that the AI never selects an illegal column.

“The prompt instructs the AI to make the Connect Four AI smarter by making it prioritize winning moves, then blocking the player's winning moves, and otherwise choosing a legal move. It also ensures the AI handles full columns safely, uses temporary move simulation correctly, and does not change the existing project structure.”

PROMPT 4:

Continue working on the SAME Connect Four project.

TASK 4 — ADD MOVE-LEVEL FEEDBACK

Add concise feedback whenever a disc is successfully placed on the board.

Requirements:

- Feedback should occur exactly once for each actual successful move.
- It should happen for both the human player and AI.
- It must NOT be printed once for every cell inspected during winner detection.
- It must NOT be printed multiple times because of AI simulation or win checking.
- Failed moves must not produce successful-placement feedback.
- Keep the message concise and suitable for a terminal game.

For example, a successful move could produce a message indicating which player placed a disc and which column was used.

Keep the existing architecture and make the smallest reasonable change.

After modifying the code:

1. Explain where the feedback was added and why it occurs exactly once.
2. Explain why AI simulations do not create fake feedback.
3. Give me a simple test procedure.
4. Do not modify unrelated functionality.

“The prompt instructs the AI to add clear move-level feedback whenever a player or the AI successfully places a disc. It ensures the message appears exactly once per valid move, does not appear for failed moves or AI simulations, and keeps the existing project structure unchanged.”